

ECOM EXPERTS

Migracion HestiaCP a HestiaCP

Manual de uso y guía de migracion

ecom_hestiamigation 1.0 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Las opciones, una a una

5. Cómo se usa, día a día

El camino recomendado: fase a fase

Prueba primero con un cliente

Qué hace cada fase

Las confirmaciones

6. Qué queda por hacer a mano después

7. Preguntas frecuentes y problemas

8. Seguridad

9. Ficha técnica

Herramienta de línea de órdenes para mudar un servidor HestiaCP entero a otro VPS.

1. Qué hace y para quién

Cuando hay que cambiar de VPS, HestiaCP no ofrece un único botón de «múdame todo». Lo que sí trae son las piezas sueltas: un comando que empaqueta un usuario del panel en un `.tar` (`v-backup-user`) y otro que lo devuelve a la vida en el servidor nuevo (`v-restore-user`). El trabajo aburrido y peligroso está en el pegamento: sacar la lista de usuarios, generar la copia de cada uno, llevarla al otro servidor, restaurarla en el orden correcto y comprobar después que no se ha quedado nada por el camino.

Este guion es ese pegamento. Se copia al **servidor viejo**, se ejecuta ahí, y él solo empuja todo al **servidor nuevo** por SSH. Está pensado para el caso más habitual: un VPS en producción, con varios clientes alojados, que hay que mover a una máquina recién instalada donde solo existe el usuario `admin`.

Además cubre tres agujeros que las copias de seguridad de HestiaCP **no** incluyen y que son la causa número uno de que una migración salga torcida: los paquetes de usuario, las plantillas propias y —sobre todo— las versiones de PHP. Si en el servidor viejo hay un dominio corriendo con PHP 8.1 y el nuevo no tiene esa versión instalada, la restauración falla o el sitio queda mal servido. El guion lo detecta antes de tocar nada y dice exactamente qué comandos hay que ejecutar para arreglarlo.

La pieza más importante, sin embargo, es la que no hace nada: **por defecto todo es una simulación**. El guion se puede lanzar entero, ver paso a paso lo que va a ocurrir, y no se toca ni un fichero hasta que se añade `--go` a mano. Y aun con `--go`, pide confirmación por teclado antes de empezar y otra vez antes de la única fase que no se puede deshacer.

2. Compatibilidad

- **HestiaCP en los dos servidores.** El guion comprueba que el origen y el destino lo son antes de seguir; si no, se para.
 - **Mismo tipo de stack web** en ambos (solo nginx, o nginx + Apache). Si difiere, avisa, porque los dominios pueden quedar mal servidos.
 - **Mismas versiones de PHP** en el destino. Las detecta y dice cuáles faltan.
 - **Bash.** Usa `set -euo pipefail`, arrays no, `local` sí: cualquier Debian o Ubuntu con los que HestiaCP funciona vale.
 - **Herramientas necesarias en el origen:** `ssh`, `rsync`, `awk`, `sed`, `df`, `tee`. Todas vienen de serie en una instalación de HestiaCP.
 - **Acceso:** clave SSH del `root` del origen autorizada en el `root` del destino. El guion usa `BatchMode`, así que si la clave no está puesta falla al instante en vez de quedarse colgado pidiendo una contraseña.
 - **No tiene interfaz gráfica.** Todo ocurre en la consola del servidor y en el fichero de registro. Por eso este manual no lleva capturas de pantalla.
-

3. Instalación

No hay instalador: es un fichero.

```
# En el servidor ORIGEN (el viejo), como root:
scp migrate-hestiacp.sh root@IP_ORIGEN:/root/
ssh root@IP_ORIGEN
chmod +x /root/migrate-hestiacp.sh
```

Antes de la primera ejecución hacen falta cuatro cosas:

1. **HestiaCP ya instalado en el destino**, con el mismo stack web que el origen y, a poder ser, la misma versión de Hestia.
2. **Las mismas versiones de PHP en el destino**. Si no se sabe cuáles son, se lanza la fase `--check` y el guion las lista y da los comandos `v-add-web-php X.Y` que faltan.
3. **Clave SSH del origen al destino**: `ssh-copy-id -p 22 root@IP_DESTINO`.
4. **Espacio libre en /backup** en los dos servidores. La fase `--check` enseña el `df -h` de ambos para poder mirarlo de un vistazo.

Qué crea: solo una carpeta `./logs/` junto al guion, con un fichero `migracion-AAAAMDD-HHMMSS.log` por ejecución. La carpeta se crea con permisos `700` y cada registro con `600`, porque dentro van nombres de usuario del panel, dominios e IPs.

Qué pasa al quitarlo: se borra el fichero `.sh` y ya está. No instala servicios, no toca el arranque, no deja cron ni entradas en el panel.

4. Las opciones, una a una

Opción	Qué hace de verdad
<code>--host <ip></code>	Servidor destino . Es la única obligatoria. Solo admite letras, dígitos y <code>. : _ -</code> ; cualquier otra cosa se rechaza.
<code>--port <n></code>	Puerto SSH del destino. Por omisión <code>22</code> . Tiene que ser un número entre 1 y 65535.
<code>--user <u></code>	Usuario SSH del destino. Por omisión <code>root</code> , y en la práctica tiene que ser root: <code>v-restore-user</code> crea usuarios del sistema.
<code>--exclude "a b"</code>	Usuarios del panel que no se migran. Por omisión <code>admin</code> , porque en el destino ya existe y machacarlo dejaría el panel inservible. Los usuarios excluidos ni se empaquetan ni salen del servidor viejo.
<code>--go</code>	Ejecuta de verdad. Sin esto todo es simulación , incluida la fase de restauración.
<code>--yes</code>	No pedir confirmación por teclado. Solo para ejecuciones desatendidas; con <code>--go</code> a secas el guion pregunta.
<code>-h</code> , <code>--help</code>	La ayuda.

Sobre `--go` conviene ser explícito: **la simulación es el modo normal y hay que salir de ella a propósito**. En simulación el guion imprime, línea a línea, cada orden que ejecutaría en local y cada una que lanzaría por SSH en el destino, precedidas de `DRY-RUN local:` y `DRY-RUN remoto:`. Es la forma de revisar una migración completa sin riesgo ninguno, y merece la pena hacerlo siempre antes de la de verdad.

5. Cómo se usa, día a día

El camino recomendado: fase a fase

Migrar de golpe con `--all --go` funciona, pero deja poco margen para reaccionar. En un servidor con clientes reales conviene ir por partes.

```
# 1) Comprobar. No cambia nada, se puede repetir las veces que haga falta.
./migrate-hestiacp.sh --host IP_DESTINO --check
```

Aquí sale la conexión SSH, si los dos son HestiaCP, las versiones de Hestia de cada uno, las versiones de PHP de cada uno (con la lista de las que faltan en el destino y el comando para instalarlas), el stack web de ambos, el espacio libre en `/backup` de los dos y la lista de usuarios que se van a migrar.

Este es el momento de instalar en el destino las versiones de PHP que falten. Si se sigue adelante sin hacerlo, los dominios que las usen quedarán mal.

```
# 2) Paquetes y plantillas propias.
./migrate-hestiacp.sh --host IP_DESTINO --prep --go

# 3) Generar las copias en el servidor viejo.
./migrate-hestiacp.sh --host IP_DESTINO --backup --go

# 4) Llevarlas al nuevo.
./migrate-hestiacp.sh --host IP_DESTINO --transfer --go

# 5) Restaurar. La única fase irreversible.
./migrate-hestiacp.sh --host IP_DESTINO --restore --go

# 6) Comparar el resultado.
./migrate-hestiacp.sh --host IP_DESTINO --verify
```

Prueba primero con un cliente

Antes de mover a todo el mundo, conviene hacer el recorrido completo con **un solo usuario** y comprobar que su sitio y su correo funcionan en el destino. Se consigue excluyendo a todos los demás:

```
./migrate-hestiacp.sh --host IP_DESTINO --exclude "admin cliente2 cliente3" --all --go
```

Qué hace cada fase

Fase	En qué servidor actúa	Qué toca
check	Solo lee, en los dos	Nada
prep	Escribe en el destino	<code>/usr/local/hestia/data/packages/</code> y <code>/usr/local/hestia/data/templates/</code>
backup	Escribe en el origen	Crea los <code>.tar</code> en <code>/backup/</code>
transfer	Lee del origen, escribe en el destino	Copia los <code>.tar</code> a <code>/backup/</code> del destino
restore	Escribe en el destino	Crea usuarios del sistema, dominios, buzones y bases de datos
verify	Solo lee, en los dos	Nada

Las confirmaciones

Con `--go`, el guion se para y pide que se escriba `SI` en mayúsculas:

1. **Al arrancar**, recordando contra qué servidor va a actuar.
2. **Antes de prep**, porque sobrescribe los paquetes y las plantillas del destino.
3. **Antes de restore**, porque `v-restore-user` machaca cualquier usuario, dominio, buzón o base de datos que ya exista en el destino con ese nombre, y eso no se deshace solo.

Si se ejecuta sin terminal —desde cron, desde una tubería, desde un guion— **el programa se niega a actuar** en vez de arrasar el destino en silencio. Para esos casos está `--yes`, que es una decisión consciente y no un descuido.

6. Qué queda por hacer a mano después

Esto **no** viaja en las copias de HestiaCP y hay que resolverlo en el destino:

- **DNS.** Cambiar los registros A o los servidores de nombres al servidor nuevo. Se hace **al final**, cuando ya se ha verificado que todo restauró bien. Mientras tanto el servidor viejo sigue sirviendo, que es justo lo que se quiere.
- **Let's Encrypt.** Los certificados de los dominios web **sí** van dentro del `.tar` y se restauran: el sitio sirve HTTPS con el mismo certificado desde el primer momento. Lo que no funciona hasta que el DNS apunte al servidor nuevo es la **renovación automática**, porque el reto HTTP-01 exige que el dominio resuelva a la IP nueva. Después de mover el DNS se puede forzar con `v-add-letsencrypt-domain <usuario> <dominio>`.
- **SSL del correo.** Hay un fallo conocido de HestiaCP por el que el certificado de los subdominios `mail.dominio` a veces no se restaura bien aunque los ficheros estén en el `.tar`. Si el correo por SSL falla tras migrar, se vuelve a emitir con el mismo comando.
- **Certificado del nombre de máquina del panel.** Es del servidor, no de un usuario, así que no va en ninguna copia: `v-add-letsencrypt-host` cuando el nombre resuelva a la IP nueva.
- **DKIM y SPF.** Las claves DKIM sí van en la copia, pero hay que revisar que los registros DNS apunten a la IP nueva.
- **Cortafuegos, fail2ban y ajustes finos** de `my.cnf`, `php.ini` o `nginx`. Si había reglas o ajustes propios, se copian a mano.
- **Probar el acceso al panel de cada usuario** y navegar por los sitios **antes** de apagar el servidor viejo.

7. Preguntas frecuentes y problemas

¿Se migra el correo de verdad, con los mensajes dentro? Sí. Las copias de usuario de HestiaCP incluyen las cuentas de correo con los buzones completos, además de dominios web con sus ficheros y configuraciones, bases de datos, zonas DNS y tareas programadas.

¿Y si el destino ya tiene datos? `v-restore-user` los sobrescribe. Por eso la fase `restore` pide confirmación y por eso el guion está pensado para un destino recién instalado, con solo el usuario `admin`. Si el destino no está limpio, hay que hacerle una copia antes.

Se ha cortado la transferencia a medias, ¿pasa algo? No. Se relanza `--transfer --go` y `rsync` continúa por donde iba. Los trozos incompletos se guardan en una carpeta aparte y nunca llegan a tener el nombre definitivo de un `.tar`, así que la fase de restauración no puede confundirse y coger una copia a medio bajar.

¿Por qué no se migra el usuario `admin`? Porque en el destino ya existe: restaurarlo encima dejaría el panel del servidor nuevo inservible. Si el `admin` del origen aloja dominios propios, hay que moverlos a otro usuario antes de migrar, o restaurarlo a mano y con cuidado.

No hay SSH sin password al destino, o el destino no es HestiaCP O falta `ssh-copy-id -p PUERTO root@IP_DESTINO`, o el puerto no es el que se ha indicado con `--port`, o en el destino todavía no está instalado HestiaCP.

Hace falta confirmación y no hay terminal Se ha lanzado con `--go` desde cron o desde una tubería. Es a propósito. Si de verdad se quiere desatendido, se añade `--yes`.

El valor de `--host` solo admite letras, dígitos y `. : _ -` Se ha colado algo raro en la IP: comillas, un espacio, un punto y coma. Es la validación de seguridad haciendo su trabajo.

No hay ningún backup que transferir Falta ejecutar antes la fase `--backup --go`.

Las versiones de Hestia no coinciden, ¿es grave? El guion avisa pero no se para. Restaurar en una versión distinta suele funcionar, pero es la primera sospechosa si algo sale raro. Lo recomendable es igualarlas antes.

Si el origen usa copias `restic` La clave de cifrado está en `/usr/local/hestia/data/users/<usuario>/restic.conf` y conviene guardarla aparte. Este guion no la necesita: usa copias `.tar` locales, que son autocontenidas.

8. Seguridad

Revisión hecha sobre la versión de julio de 2026 del guion. Se buscaron credenciales escritas dentro, órdenes destructivas sin confirmación, órdenes construidas con variables sin comillas, descargas del tipo `curl` | `bash`, ficheros temporales con permisos abiertos, y volcados de SQL o copias con datos reales de clientes dentro de la carpeta.

No había ninguna credencial ni ningún dato real de cliente en el proyecto. Tampoco `curl` | `bash`, ni ficheros temporales, ni volcados de base de datos. La carpeta contiene únicamente el guion y su documentación.

Sí aparecieron siete problemas, todos corregidos:

1. Ejecución de órdenes arbitrarias a través de los parámetros (grave). `migrate-hestiacp.sh`, funciones `run()` y `rrun()`. Las órdenes se montan como texto y se pasan por `eval` en el servidor origen, y por SSH como `root` en el destino. Los valores de `--host`, `--port` y `--user` se incrustaban ahí sin comprobar nada, así que un valor con comillas y un punto y coma ejecutaba lo que quisiera en los **dos** servidores. Se comprobó en pruebas: la orden inyectada se ejecutaba. Corregido con una validación estricta por lista blanca de caracteres justo después de leer los argumentos; cualquier valor con algo que no sea una letra, un dígito o `. : _ -` se rechaza antes de que el guion llegue a hacer nada.

2. Escalada de privilegios a root en el servidor destino (grave). `migrate-hestiacp.sh`, fase `prep`. Tras copiar los datos, hacía `chown -R admin:admin` sobre `/usr/local/hestia/data/packages` y `.../templates`. HestiaCP **ejecuta esas plantillas como root** cada vez que crea o reconstruye un dominio, de modo que

cualquiera con acceso al usuario `admin` del panel podía editar una plantilla y obtener root en el servidor nuevo. Corregido a `chown -R root:root`, que es como las deja HestiaCP.

3. Copias de clientes excluidos enviadas igualmente al servidor nuevo (fuga de datos). `migrate-hestiacp.sh`, fase `transfer`. Copiaba `/backup/*.tar` en bloque, de forma que los `.tar` de los usuarios excluidos con `--exclude` —con su correo real, sus bases de datos y los ficheros de sus clientes dentro— salían del servidor viejo igual que los demás, además de todas las copias antiguas acumuladas. Corregido: ahora se selecciona explícitamente el `.tar` más reciente de cada usuario que sí se migra, se avisa de los que no tienen copia y se aborta si no hay ninguna que enviar.

4. Restauración silenciosa de una copia truncada (pérdida de datos). `migrate-hestiacp.sh`, fase `transfer`. Usaba `rsync --partial`, que al cortarse deja el trozo descargado **con el nombre definitivo**. La fase `restore` elige el `.tar` más reciente de cada usuario, así que podía coger ese fichero a medias y restaurar una copia corrupta sin dar un solo aviso. Corregido con `--partial-dir=.rsync-partial`: el trozo va a una carpeta aparte y solo pasa a tener el nombre bueno cuando está entero. Se sigue pudiendo reanudar.

5. Fases destructivas sin ninguna confirmación. `migrate-hestiacp.sh`. Con `--all --go`, el guion generaba las copias, las transfería y restauraba encima del destino de un tirón, sin preguntar. Corregido con una confirmación por teclado al arrancar, otra antes de `prep` (sobrescribe plantillas del destino) y otra antes de `restore` (sobrescribe usuarios, dominios, buzones y bases de datos). Si no hay terminal —cron, tubería—, se niega a actuar; `--yes` permite el desatendido a propósito.

6. Actualización de Hestia sin supervisión en mitad de la migración. `migrate-hestiacp.sh`, fase `prep`. Lanzaba `v-update-sys-hestia-all` en el destino con la salida silenciada. Eso actualiza HestiaCP entero, sin vigilancia, en medio de la migración, y podía dejar el destino en una versión distinta de la del origen: justo lo que la fase `check` exige que coincida. Retirado; la corrección de permisos que era su motivo la hace ya el `chown`.

7. Registro legible por cualquiera y `restore` con el nombre de fichero sin comillas. El registro guarda usuarios del panel, dominios e IPs y se creaba con los permisos por omisión: ahora la carpeta se crea `700` y el fichero `600`. Y en la fase `restore`, el nombre del `.tar` se obtiene con un `ls` **en el servidor remoto** y se volvía a inyectar sin comillas en una orden SSH ejecutada como root; ahora se valida que sea un nombre de fichero y se pasa entrecomillado.

Además, tres arreglos de robustez que no son de seguridad pero evitan sustos: `--host` sin valor moría con un `unbound variable` incomprensible y ahora dice qué falta; las conexiones SSH usan `BatchMode` y `ConnectTimeout`, de modo que una clave sin autorizar o una IP mal escrita fallan al instante en vez de quedarse colgadas pidiendo contraseña; y la fase `check` muestra ahora el espacio libre en `/backup` de los dos servidores, porque `v-backup-user` escribe en el servidor **de producción** y llenarle el disco lo tumba antes de haber migrado nada.

Qué queda pendiente, a propósito. El guion sigue usando `eval` en `run()`. Es necesario: las órdenes llevan comillas internas (`-e 'ssh -p N'`) que hay que respetar, y reescribirlo con arrays cambiaría el comportamiento de un programa que corre como root en servidores de producción. La mitigación elegida es que **nada que venga de fuera llega hasta ahí sin validar**, que es la corrección número 1. Queda documentado y comentado en el propio código.

Tampoco se ha automatizado la copia de seguridad del destino antes de restaurar: se avisa en la confirmación y en este manual, pero es una decisión del operador, no del guion.

9. Ficha técnica

Ficheros del proyecto

Fichero	Qué es
<code>migrate-hestiacp.sh</code>	El guion. Bash, <code>set -euo pipefail</code> . Único ejecutable.
<code>README.md</code>	Resumen y uso rápido.
<code>GUIA-migracion-hestiacp.md</code>	Guía larga: método manual equivalente, SSL y pasos posteriores.

Rutas que toca

Ruta	Servidor	Uso
<code>/usr/local/hestia/bin/</code>	ambos	<code>v-list-users</code> , <code>v-backup-user</code> , <code>v-restore-user</code> , <code>v-list-web-domains</code>
<code>/usr/local/hestia/conf/hestia.conf</code>	ambos	Solo lectura: versión, <code>WEB_SYSTEM</code> , <code>PROXY_SYSTEM</code>
<code>/usr/local/hestia/data/packages/</code>	destino	Se sobrescribe en la fase <code>prep</code>
<code>/usr/local/hestia/data/templates/</code>	destino	Se sobrescribe en la fase <code>prep</code>
<code>/etc/php/</code>	ambos	Solo lectura: detectar las versiones de PHP instaladas
<code>/backup/</code>	ambos	Origen de los <code>.tar</code> ; destino donde se dejan
<code>./logs/migracion-*.log</code>	origen	Registro de cada ejecución. Carpeta <code>700</code> , fichero <code>600</code>

Comandos de HestiaCP que se usan

`v-list-users`, `v-list-web-domains`, `v-backup-user`, `v-restore-user`. Y como sugerencias para el operador, sin ejecutarlos: `v-add-web-php`, `v-add-letsencrypt-domain`, `v-add-letsencrypt-host`.

Herramientas externas: `ssh` (con `BatchMode=yes` y `ConnectTimeout=10`), `rsync` (`-az`, `--info=progress2`, `-partial-dir=.rsync-partial`), `awk`, `sed`, `df`, `tee`, `chmod`.

Sin cron, sin servicios, sin base de datos. El guion se ejecuta a mano y termina.

Método manual equivalente, si algún día hay que hacerlo sin el guion:

```
# En el ORIGEN:
v-backup-users
rsync -az -e ssh /backup/*.tar root@IP_DESTINO:/backup/

# En el DESTINO:
for i in $(ls -c1 /backup/*.tar | sed -E "s#^/backup/(.+)\.+tar#\1#" | sort -u); do
  fbackup="$(ls -rt -c1 /backup/$i.*tar | tail -n1 | awk -F/ '{print $NF}')"
  /usr/local/hestia/bin/v-restore-user "$i" "$fbackup"
done
```

Ojo: ese `rsync /backup/*.tar` es exactamente el problema número 3 de la revisión de seguridad. Si se hace a mano, hay que nombrar los `.tar` uno a uno.

Ecom Experts · 2026 · Licencia AFL-3.0